

Trabalho: Jogo da Cobrinha no CPULator

Desenvolvido por: Elis Rodrigues Borges – 231018875

Link para repositório no GitHub com o código-fonte: <https://github.com/elisrb/cobrinha-cpulator>

Visão Geral

O trabalho é uma réplica do clássico jogo da cobrinha na linguagem C, para ser executado no [simulador ARMv7 \(De1-SoC\) no CPULator](#).

A cobra se desloca em direções ortogonais, virando 90 graus à esquerda ou direita sob o comando das teclas. Maçãs vermelhas surgem na tela (uma de cada vez) em posições aleatórias. Quando a cabeça da cobra encontra uma maçã (come a maçã): a maçã desaparece; a cobra aumenta uma unidade em seu comprimento; e outra maçã é gerada aleatoriamente.

Se a cabeça da cobra atingir o seu corpo, ela morre e encerra o jogo. Se atingir o limite da tela, ela rebate para o lado oposto.

Como jogar

1. Cole o código no [simulador ARMv7 \(De1-SoC\) no CPULator](#)
2. Clique em *Compile and Load (F5)* e, em seguida, em *Continue (F3)*
3. O jogo deve aparecer na seção **VGA pixel buffer** no painel **Devices** (a imagem pode ser ampliada clicando na seta para baixo na seção e aumentando o zoom)
4. Clique no campo *Type here:* na seção **PS/2 keyboard or mouse (IRQ 79 ff200100)** no painel **Devices** para ativar o cursor
5. Use as setas para virar a cobrinha para esquerda ou direita

Implementação

O projeto foi desenvolvido em linguagem C nativa (*bare-metal*) focado no processador ARMv7 da DE1-SoC, manipulando buffers de vídeo mapeados em memória e periféricos digitais. As bibliotecas utilizadas foram:

stdint.h: suporte para o tipo *uint16_t* usado na matriz da tela VGA e nos sprites.

stdio.h: suporte para o tipo *size_t* usado internamente por bibliotecas e funções de sistema.

stdlib.h: contém as funções usadas para sortear a posição da maçã.

O controle de hardware é efetuado por meio de Entrada e Saída Mapeada em Memória (MMIO), sendo necessário definir ponteiros para os dispositivos utilizados:

Subsistema de Vídeo (VGA_BASE): Mapeado no endereço 0xC8000000. O buffer de exibição é mapeado por meio de um ponteiro para uma matriz *uint16_t*. A tela opera na resolução de 320x240 pixels (WIDTH e HEIGHT). No entanto, o controlador VGA adota uma largura de 512 pixels (LWIDTH), portanto, a escrita correta de um pixel exige ignorar as colunas invisíveis.

Interface de Teclado (FPGA_PS2_KEYBOARD): Mapeado no endereço 0xFF200100. O acesso a este registrador de 32 bits permite a leitura de uma fila interna de dados (FIFO) que descrevem as teclas pressionadas ou soltas.

Temporizador: Mapeado no endereço 0xFFFFEC600. O valor atual do timer é usado no código como *seed* para a geração de números aleatórios.

A interface visual do jogo é segmentada em uma malha (grid) de blocos, em que cada bloco é uma matriz de 9x9 pixels (TILE_SIZE). Esse tamanho foi escolhido por permitir que haja um pixel central, o que facilita o desenho dos sprites da cobra e da maçã. Esses sprites foram declarados globalmente como matrizes constantes 9x9 em que cada elemento é um código RGB565 (*uint16_t*). As funções relativas à interface gráfica são:

tela_fundo: Inicializa o cenário no início do jogo, mostrando cada bloco do grid. A função varre todos os pixels da tela gerando o padrão de xadrez: se o índice da linha ou coluna for múltiplo de (TILE_SIZE + 1), o pixel recebe a cor de delimitação do grid (GRID_COLOR); caso contrário, recebe a cor de fundo do cenário (BG_COLOR).

show_pixel: Recebe coordenadas de linha (int) e coluna (int) e uma cor codificada em RGB565 (*uint16_t*), e muda a cor do pixel nas coordenadas fornecidas de acordo com a cor fornecida. A função valida as coordenadas e avalia se o pixel possui a cor TRANS (0x8001); caso positivo, a escrita é ignorada, possibilitando o efeito de transparência.

show_tile: Recebe coordenadas de linha (int) e coluna (int) e um sprite (matriz 9x9 de *uint16_t*), e escreve o sprite na posição correspondente no grid. As coordenadas são referentes à posição do bloco no grid, que a função converte em coordenadas de pixels na tela através da função matemática: [pixel_coord = grid_coord * (TILE_SIZE + 1) + 1]. Tendo as coordenadas adequadas, a função *show_pixel* é chamada para cada pixel do sprite fornecido.

clear_tile: Recebe coordenadas de linha (int) e coluna (int) no grid. Tem o mesmo funcionamento de *show_tile*, mas preenche a região do bloco com a cor de fundo (BG_COLOR).

A função **keyboard_input** realiza a leitura da interface PS/2 keyboard. Um laço *while* avalia o bit 15 (0x8000) do registrador do PS/2 para identificar se há dados válidos na fila. Enquanto uma tecla está pressionada, o teclado transmite o byte correspondente (sinal *make*), e quando ela é solta, o teclado transmite o byte de escape 0xF0 (sinal *break*). A função rastreia esse comportamento para garantir que, se o usuário mantiver o dedo pressionado na tecla, a cobra muda de direção apenas uma única vez.

A função **delay** força a execução de várias operações nulas em assembly (*nop*) para permitir um atraso controlado no movimento da cobra.

As principais estruturas de dados usadas para o controle são variáveis *int*:

snake_size: tamanho da cobra, inicializado em 5.

direction: direção atual da cabeça da cobra (0 = esquerda, 1 = cima, 2 = direita, 3 = baixo), inicializado em 2.

pos e **old_pos:** matrizes 100 (MAX_SNAKE_SIZE) x 2, armazenam as coordenadas (linha e coluna no grid) de cada bloco que compõe a cobra, da cabeça (índice 0) à cauda (índice *snake_size - 1*). *pos* armazena a posição atual e *old_pos* a posição anterior.

maca_l e **maca_c:** as coordenadas atuais da maçã (linha e coluna no grid).

A função **show_snake** recebe *pos*, *snake_size* e *direction*, e chama a função *show_tile* para mostrar os sprites adequados (cabeça, corpo e cauda) nas posições e direções adequadas.

A função **gerar_maca** gera uma nova posição aleatória para a maçã e usa as posições atuais da cobra para validar. Se a posição gerada para a maçã coincide com uma das posições da cobra, gera novamente.

A função **main** gerencia a inicialização e a lógica do jogo, e chama as demais funções descritas acima. A flag **perdeu** é inicializada em 0 e é usada para marcar se o jogo continua (0) ou se o jogador perdeu (1). O fluxo do loop principal do jogo é:

Enquanto *perdeu* = 0:

1. Chamada de *show_snake* para imprimir a cobra na posição atual;
2. Chamada de *show_tile* para imprimir a maçã na posição atual;
3. Chamada de *keyboard_input* para ler se alguma das setas foi pressionada;
4. Chamada de *delay* para atrasar o movimento da cobra;
5. Um *switch* calcula a direção da cobra dependendo do input recebido;
6. Salva os valores de *pos* em *old_pos*;
7. Um *switch* calcula a próxima posição da cabeça da cobra dependendo da direção atual;
8. A posição dos demais blocos que compõem o corpo da cobra é atualizada, de forma que cada bloco passa a ocupar a última posição do seu antecessor;
9. Chamada de *clear_tile* para apagar a última posição da cauda;
10. Um *if* checa se a cabeça colidiu com o corpo (caso positivo, o valor de *perdeu* se torna 1);
11. Um *if* checa se a cabeça colidiu com a maçã (caso positivo, incrementa o tamanho da cobra em 1, e chama *gera_maca*).

Imagens do jogo

